```
# Read JSON file
df_json = spark.read.json("people.json")
print("DataFrame from JSON:")
df_json.show()
df_json.printSchema()

# For Parquet, you'd first write a DataFrame to Parquet, then read it back
# df_csv.write.mode("overwrite").parquet("people.parquet")
# df_parquet = spark.read.parquet("people.parquet")
# print("\nDataFrame from Parquet:")
# df_parquet.show()
# df_parquet.printSchema()

spark.stop()
```

Basic operations:

Once you have a DataFrame, you can perform various fundamental operations.

select

`select()` is used to select columns from a DataFrame. You can select specific columns, or use expressions.

Example (Python):

```
Python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("DFBasicOps").getOrCreate()

data = [("Alice", 30, "New York"), ("Bob", 25, "London"), ("Charlie", 35, "Paris")]
columns = ["Name", "Age", "City"]
df = spark.createDataFrame(data, columns)
df.show()

# Select a single column
df.select("Name").show()

# Select multiple columns
df.select("Name", "City").show()

# Select columns using col() function (recommended for robustness)
```

```
df.select(col("Name"), col("Age")).show()

# Select a column and alias it
df.select(df.Name.alias("Full_Name"), df.Age).show()

spark.stop()
```

---

filter

`filter()` (or `where()`) is used to filter rows based on a given condition. It returns a new DataFrame containing only the rows that satisfy the condition.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("DFBasicOps").getOrCreate()

data = [("Alice", 30, "New York"), ("Bob", 25, "London"), ("Charlie", 35, "Paris")]
columns = ["Name", "Age", "City"]
df = spark.createDataFrame(data, columns)
df.show()

# Filter using string expression
df.filter("Age > 28").show()

# Filter using column object
df.filter(col("Age") <= 30).show()

# Filter with multiple conditions (AND)
df.filter((col("Age") > 25) & (col("City") == "New York")).show()

# Filter with multiple conditions (OR)
df.filter((col("Age") < 25) | (col("City") == "Paris")).show()

spark.stop()
```

---

where

`where()` is an alias for `filter()`. They behave identically.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("DFBasicOps").getOrCreate()

data = [("Alice", 30, "New York"), ("Bob", 25, "London"), ("Charlie", 35, "Paris")]
columns = ["Name", "Age", "City"]
df = spark.createDataFrame(data, columns)
df.show()

# Using where()
df.where(col("City") == "London").show()

spark.stop()
```

---

show

show() displays the contents of the DataFrame in a tabular format. It's very useful for inspecting your data.

Example (Python):

Python

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("DFBasicOps").getOrCreate()

data = [("Alice", 30, "New York"), ("Bob", 25, "London"), ("Charlie", 35, "Paris")]
columns = ["Name", "Age", "City"]
df = spark.createDataFrame(data, columns)

# Show entire DataFrame
print("Entire DataFrame:")
df.show()

# Show first N rows
print("First 2 rows:")
df.show(2)

# Show without truncating column values
print("DataFrame without truncating:")
```

```
df.show(truncate=False)

spark.stop()
```

---

printSchema

printSchema() displays the schema (column names and their data types) of the DataFrame in a tree-like format. This is crucial for understanding your data's structure and for debugging.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder.appName("DFBasicOps").getOrCreate()

data = [("Alice", 30, "New York"), ("Bob", 25, "London", ["reading", "cycling"]), ("Charlie", 35, "Paris", [])]
schema = StructType([
    StructField("Name", StringType(), True),
    StructField("Age", IntegerType(), True),
    StructField("City", StringType(), True),
    StructField("Hobbies", ArrayType(StringType()), True)
])
df = spark.createDataFrame(data, schema=schema)

print("DataFrame Schema:")
df.printSchema()

spark.stop()
```

DataFrame Transformations

DataFrame transformations are operations that return a new DataFrame without modifying the original. They are **lazy** - meaning they are not executed until an **action** (like show(), count(), collect(), write()) is called.

withColumn, drop, alias, selectExpr, rename

withColumn

`withColumn()` is used to add a new column or replace an existing column in a DataFrame.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit

spark = SparkSession.builder.appName("DFTransformations").getOrCreate()

data = [("Alice", 30), ("Bob", 25), ("Charlie", 35)]
columns = ["Name", "Age"]
df = spark.createDataFrame(data, columns)
df.show()

# Add a new column 'City' with a literal value
df_with_city = df.withColumn("City", lit("Unknown"))
print("After adding 'City' column:")
df_with_city.show()

# Add a new column 'Age_in_Months' based on an existing column
df_with_months = df.withColumn("Age_in_Months", col("Age") * 12)
print("After adding 'Age_in_Months' column:")
df_with_months.show()

# Replace an existing column (e.g., convert 'Age' to StringType)
df_replaced_age = df.withColumn("Age", col("Age").cast("string"))
print("After replacing 'Age' column type:")
df_replaced_age.printSchema()
df_replaced_age.show()

spark.stop()
```

---

drop

`drop()` is used to remove one or more columns from a DataFrame.

Example (Python):

Python

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("DFTransformations").getOrCreate()

```

```
data = [("Alice", 30, "New York"), ("Bob", 25, "London"), ("Charlie", 35, "Paris")]
columns = ["Name", "Age", "City"]
df = spark.createDataFrame(data, columns)
df.show()

# Drop a single column
df_no_city = df.drop("City")
print("After dropping 'City' column:")
df_no_city.show()

# Drop multiple columns
df_no_age_city = df.drop("Age", "City")
print("After dropping 'Age' and 'City' columns:")
df_no_age_city.show()

spark.stop()
```

---

alias

`alias()` is used to rename a column or an expression. It's often used within `select` or `withColumn`.

Example (Python):

```
Python
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("DFTransformations").getOrCreate()

data = [("Alice", 30), ("Bob", 25)]
columns = ["Name", "Age"]
df = spark.createDataFrame(data, columns)
df.show()

# Using alias() in select
df.select(col("Name").alias("Full_Name"), col("Age")).show()

# Using alias() with an expression in select
df.select((col("Age") * 2).alias("Double_Age")).show()

# Using alias() with withColumn (less common as withColumnRenamed is more direct)
df_aliased_age = df.withColumn("Age", col("Age").alias("YearsOld")) # This will actually just rename the column 'Age' to 'YearsOld' if it's a new column, or if it's replacing it, it will create a new column named 'Age' with the content of 'YearsOld'
```

```
print("Using alias with withColumn (results in a new column if original name is kept):")
df_aliased_age.show()
df_aliased_age.printSchema()

spark.stop()
```

---

selectExpr

selectExpr() allows you to use SQL-like expressions to select columns and apply transformations directly. It's very convenient for complex expressions.

Example (Python):

Python

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("DFTransformations").getOrCreate()

data = [("Alice", 30, "New York"), ("Bob", 25, "London")]
columns = ["Name", "Age", "City"]
df = spark.createDataFrame(data, columns)
df.show()

# Select specific columns and rename
df.selectExpr("Name", "Age as Person_Age", "City").show()

# Perform calculations using SQL expressions
df.selectExpr("Name", "Age * 12 as Age_in_Months", "upper(City) as City_Upper").show()

# Apply conditional logic
df.selectExpr("Name", "CASE WHEN Age > 28 THEN 'Adult' ELSE 'Young' END as Category").show()

spark.stop()
```

---

rename

Spark DataFrames don't have a direct rename method for columns like Pandas. Instead, you typically use withColumnRenamed() or select() with alias().

withColumnRenamed() is the most common and direct way to rename a single column.

Example (Python):

Python

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("DFTransformations").getOrCreate()

data = [("Alice", 30), ("Bob", 25)]
columns = ["Name", "Age"]
df = spark.createDataFrame(data, columns)
df.show()

# Rename a single column using withColumnRenamed
df_renamed = df.withColumnRenamed("Name", "Full_Name")
print("After renaming 'Name' to 'Full_Name':")
df_renamed.show()
df_renamed.printSchema()

# Renaming multiple columns (chaining or using select)
# Chaining withColumnRenamed
df_renamed_multiple = df.withColumnRenamed("Name",
"PersonName").withColumnRenamed("Age", "PersonAge")
print("After renaming multiple columns (chaining):")
df_renamed_multiple.show()

# Using select with alias for multiple renames
df_renamed_select = df.select(
    df.Name.alias("Person_Name"),
    df.Age.alias("Person_Age")
)
print("After renaming multiple columns (using select with alias):")
df_renamed_select.show()

spark.stop()
```

Chaining transformations

One of the most powerful features of Spark DataFrames is the ability to **chain transformations**. This makes your code more concise, readable, and allows Spark to optimize the entire sequence of operations more effectively due to lazy evaluation.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit

spark = SparkSession.builder.appName("ChainingTransformations").getOrCreate()

data = [("Alice", 30, "NY", 75000),
        ("Bob", 25, "LD", 60000),
        ("Charlie", 35, "NY", 90000),
        ("David", 22, "SF", 55000),
        ("Eve", 40, "LD", 100000)]
columns = ["Name", "Age", "City_Code", "Salary"]
df = spark.createDataFrame(data, columns)
df.show()

# Chain multiple transformations:
# 1. Filter for age > 25
# 2. Add a new column 'Bonus'
# 3. Select specific columns and rename one
# 4. Show the result
df_processed = df.filter(col("Age") > 25) \
                 .withColumn("Bonus", col("Salary") * 0.10) \
                 .select(col("Name"),
                         col("Age").alias("YearsOld"),
                         col("Salary"),
                         col("Bonus"),
                         lit("Processed").alias("Status")) \
                 .sort(col("Salary").desc()) # Add a sort for good measure

print("Chained transformations result:")
df_processed.show()

spark.stop()
```

Creating calculated columns

Calculated columns are new columns derived from existing columns using expressions or functions.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, concat_ws, lit, when

```